

Министерство цифрового развития государственного управления,
информационных технологий и связи Республики Татарстан
государственное автономное профессиональное образовательное учреждение
«Международный центр компетенций –
Казанский техникум информационных технологий и связи»

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ ПОДГОТОВКЕ
(УЧЕБНОЙ ПРАКТИКЕ)

ПМ.01 УП.01.01 РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО
ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ
(наименование профессионального модуля)

Выполнил обучающийся Смирнов Владимир Викторович
(ФИО)

Группа 320 специальность 09.02.07 «Информационные системы и
программирование»

Оценка ____ (_____)

Начало практики: 20.04.2026 г.

Гаизов К.А.
(Ф.И.О. и подпись руководителя практики)

Фролов А.В.
(Ф.И.О. и подпись руководителя практики)

Окончание практики: 09.05.2026 г.

Гаизов К.А.
(Ф.И.О. и подпись руководителя практики)

Фролов А.В.
(Ф.И.О. и подпись руководителя практики)

М.П.

Казань, 2026 г.

СОДЕРЖАНИЕ

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА ТЕМУ: Веб-приложение «VetClinic»	3
1 ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ РАЗРАБОТКИ	6
1.1 Обоснование выбора языка программирования	6
1.2 Обоснование выбора среды разработки	6
1.3 Обоснование выбора СУБД	7
2 РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА	9
2.1 Анализ предметной области.....	9
2.2 Проектирование базы данных.....	10
2.3 Реализация интерфейсов и функционала	13
2.4 Тестирование программного продукта	17
2.5 Руководство пользователя	19
ЗАКЛЮЧЕНИЕ	24

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА ТЕМУ: Веб-приложение «VetClinic»

Индивидуальное задание предполагает разработку веб-приложения для управления ветеринарной клиникой с использованием технологии Blazor WebAssembly и ASP.NET Core Web API на языке программирования C#, а также интеграцию с СУБД PostgreSQL для автоматизации процессов учета владельцев питомцев, животных, записей на прием, медицинских карт, вакцинаций, складских операций, финансов и уведомлений.

В рамках проекта необходимо спроектировать базу данных и заполнить ее тестовыми данными. Для демонстрации работы базы данных требуется реализовать REST API и клиентскую часть приложения с возможностью просмотра, добавления, изменения и удаления данных, а также формирования отчетов и отображения информации в административной панели.

Система предусматривает реализацию четырех основных ролевых моделей: администратора, ветеринарного врача, владельца питомца и ассистента, каждая из которых обладает определенным набором прав доступа и функциональных возможностей.

Администратор должен иметь следующие возможности:

- управление пользователями системы;
- просмотр и управление владельцами питомцев;
- просмотр и управление питомцами;
- управление расписанием приемов;
- управление услугами клиники и их стоимостью;
- работа со складом лекарств, вакцин, кормов и расходных материалов;
- просмотр операций и госпитализаций;
- работа с финансовыми счетами;
- формирование отчетов и аналитики;
- просмотр данных в административной панели;
- получение уведомлений о критических остатках и важных событиях.

Ветеринарный врач должен иметь следующие возможности:

- просмотр своих записей на прием;

- открытие карточек питомцев;
- создание медицинских записей;
- указание жалоб, диагноза, лечения и рекомендаций;
- просмотр истории болезней питомцев;
- добавление записей о вакцинации;
- планирование операций;
- просмотр информации о стационаре и складе;
- получение уведомлений о новых записях и изменениях расписания.

Владелец питомца должен иметь следующие возможности:

- регистрация и авторизация в системе;
- добавление и редактирование своих питомцев;
- запись питомца на прием к ветеринарному врачу;
- просмотр собственных записей на прием;
- просмотр истории болезней своих питомцев;
- просмотр вакцинаций;
- просмотр выставленных счетов;
- получение уведомлений о приемах и вакцинациях.

Ассистент должен иметь следующие возможности:

- просмотр приемов;
- работа со складскими позициями;
- списание расходных материалов;
- контроль критических остатков и сроков годности;
- просмотр активных госпитализаций;
- добавление записей ухода за питомцами в стационаре;
- получение складских и стационарных уведомлений.

В приложении должны быть реализованы основные функциональные блоки: авторизация и разграничение доступа, учет владельцев и питомцев, онлайн-запись на прием, электронные медицинские карты, вакцинации и профилактика, складской учет, операции, стационар, финансовый учет, отчеты, уведомления и административная панель.

Серверная часть приложения должна предоставлять REST API для выполнения операций с основными сущностями системы. Контроллеры принимают запросы от клиентского приложения, передают данные в сервисный слой и возвращают результат обработки. Работа с базой данных выполняется через Entity Framework Core, а для авторизации и разграничения прав используется ASP.NET Core Identity с JWT-токенами.

Клиентская часть должна быть реализована на Blazor WebAssembly. Интерфейс приложения должен содержать страницы для входа, регистрации, просмотра питомцев, создания записей на прием, работы с медицинскими картами, вакцинациями, складом, операциями, стационаром, счетами, отчетами и уведомлениями. Меню и доступные страницы должны зависеть от роли авторизованного пользователя.

База данных должна хранить сведения о пользователях, питомцах, приемах, медицинских записях, вакцинациях, складских позициях, складских операциях, операциях, госпитализациях, записях ухода, услугах клиники, счетах и уведомлениях. Между основными таблицами необходимо реализовать связи, обеспечивающие целостность данных и корректную работу приложения.

Для обновления информации в режиме реального времени необходимо использовать технологию SignalR. С ее помощью должны передаваться уведомления о новых записях на прием, предстоящих вакцинациях, критических остатках на складе и изменениях в стационаре.

Разрабатываемая система должна обеспечивать удобный пользовательский интерфейс, проверку прав доступа, корректную обработку данных и стабильную работу без аварийных завершений. Итоговое веб-приложение должно демонстрировать автоматизацию основных процессов ветеринарной клиники и быть пригодным для учебной защиты проекта.

1 ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ РАЗРАБОТКИ

1.1 Обоснование выбора языка программирования

В качестве основного языка программирования при разработке веб-приложения VetClinic выбран язык C#. Данный язык является одним из основных инструментов платформы .NET и широко применяется для создания современных веб-приложений, клиент-серверных систем и REST API.

Выбор C# обусловлен тем, что язык поддерживает строгую типизацию, объектно-ориентированный подход и асинхронное программирование. Это позволяет создавать надежные приложения с понятной структурой кода, а также снижает вероятность ошибок при работе с данными пользователей, питомцев, записей на прием, медицинских карт и других сущностей системы.

Дополнительным преимуществом является возможность использования единого технологического стека для серверной и клиентской частей приложения. Серверная часть проекта VetClinic реализуется с использованием ASP.NET Core Web API, а клиентская часть - с использованием Blazor WebAssembly. Благодаря этому значительная часть разработки выполняется на одном языке программирования, что упрощает сопровождение проекта и взаимодействие между его модулями.

Также язык C# хорошо подходит для работы с Entity Framework Core, ASP.NET Core Identity, JWT-авторизацией и SignalR. Эти технологии используются в проекте для взаимодействия с базой данных, разграничения прав доступа, защиты API и отправки уведомлений в режиме реального времени.

Таким образом, язык C# выбран как современный, надежный и удобный инструмент для разработки веб-приложения VetClinic, так как он позволяет реализовать серверную часть, клиентскую часть и бизнес-логику проекта в единой экосистеме .NET.

1.2 Обоснование выбора среды разработки

В качестве среды разработки для создания веб-приложения VetClinic использовалась Visual Studio.

Данная среда предоставляет широкий набор инструментов для разработки приложений на платформе .NET и имеет встроенную поддержку технологий ASP.NET Core, Blazor WebAssembly и Entity Framework Core.

Visual Studio обеспечивает удобную работу со структурой решения, состоящего из серверной части, клиентской части и общей библиотеки. Это позволяет разрабатывать проекты VetClinic.Api, VetClinic.Client и VetClinic.Shared в одном рабочем пространстве, быстро переходить между файлами, контроллерами, сервисами, моделями и компонентами интерфейса.

Одним из преимуществ Visual Studio является наличие встроенного отладчика. С его помощью можно проверять работу серверной логики, API-запросов, клиентских страниц и взаимодействия с базой данных. Это особенно важно при разработке системы, в которой используются авторизация, роли пользователей, запись на прием, медицинские карты, складской учет и уведомления.

Также среда разработки предоставляет удобные средства для работы с NuGet-пакетами, миграциями Entity Framework Core и системой контроля версий Git. Благодаря этому упрощается подключение необходимых библиотек, настройка базы данных, создание миграций и сопровождение проекта в процессе разработки.

Таким образом, Visual Studio выбрана как удобная и функциональная среда разработки, позволяющая эффективно создавать, отлаживать и поддерживать веб-приложение VetClinic на платформе .NET.

1.3 Обоснование выбора СУБД

В качестве системы управления базами данных для веб-приложения VetClinic выбрана PostgreSQL. Данная СУБД является современной реляционной системой хранения данных с открытым исходным кодом и широко применяется при разработке веб-приложений.

Выбор PostgreSQL обусловлен тем, что проект VetClinic работает с большим количеством связанных данных.

В базе данных необходимо хранить сведения о пользователях, ролях, владельцах питомцев, животных, записях на прием, медицинских картах, вакцинациях, складских позициях, операциях, госпитализациях, счетах и уведомлениях. Для таких данных важны поддержка связей между таблицами, целостность данных и стабильная работа запросов.

PostgreSQL поддерживает первичные и внешние ключи, ограничения целостности, транзакции и индексацию. Это позволяет обеспечить корректное хранение информации и надежную обработку операций, связанных с созданием записей на прием, добавлением медицинских записей, изменением складских остатков и формированием финансовых данных.

Дополнительным преимуществом PostgreSQL является совместимость с Entity Framework Core. Благодаря этому в проекте можно использовать подход Code First, при котором структура базы данных создается на основе моделей приложения. Это упрощает проектирование таблиц, настройку связей между сущностями и дальнейшее сопровождение базы данных с помощью миграций.

Таким образом, PostgreSQL выбрана как надежная и удобная СУБД, подходящая для хранения связанных данных веб-приложения VetClinic и эффективной работы с ними через Entity Framework Core.

2 РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА

2.1 Анализ предметной области

Предметная область проекта связана с автоматизацией работы ветеринарной клиники. Основной задачей системы является организация учета владельцев питомцев, животных, записей на прием, медицинских данных, складских остатков, финансовых операций и уведомлений в едином веб-приложении.

В рамках предметной области можно выделить несколько основных процессов:

- регистрация и авторизация пользователей;
- учет владельцев питомцев;
- учет карточек питомцев;
- запись питомцев на прием к ветеринарному врачу;
- ведение электронных медицинских карт;
- хранение истории болезней;
- планирование вакцинаций и профилактических мероприятий;
- учет лекарств, вакцин, кормов и расходных материалов;
- ведение операций и стационара;
- создание счетов и учет оплаты;
- формирование отчетов;
- отправка уведомлений пользователям.

Система предусматривает разделение пользователей по ролям. Администратор обладает расширенными правами доступа и может управлять пользователями, владельцами, питомцами, расписанием, услугами, складом, финансами и отчетами. Ветеринарный врач работает с приемами, карточками питомцев, медицинскими записями, вакцинациями и операциями. Владелец питомца может добавлять своих животных, записывать их на прием, просматривать историю болезней, вакцинации и счета. Ассистент работает со складом, списанием материалов, стационаром и записями ухода.

Основными объектами системы являются пользователь, питомец, прием, медицинская запись, вакцинация, складская позиция, складская операция, операция, госпитализация, запись ухода, услуга клиники, счет и уведомление. Между ними существуют логические связи: питомец принадлежит владельцу, прием связан с питомцем, врачом и услугой, медицинская запись создается по результатам приема, счет формируется на основании приема и выбранной услуги.

Важной особенностью предметной области является необходимость разграничения доступа к данным. Например, владелец должен видеть только своих питомцев, свои записи на прием, медицинскую историю своих животных и выставленные ему счета. Врач должен получать доступ к данным, связанным с его приемами и пациентами, а администратор должен иметь возможность контролировать работу клиники в целом.

Также для ветеринарной клиники важна своевременность получения информации. Поэтому в проекте предусмотрены уведомления о новых записях на прием, предстоящих вакцинациях, критических остатках на складе и изменениях в стационаре. Для передачи таких уведомлений используется технология SignalR, позволяющая обновлять данные в режиме реального времени.

Таким образом, разрабатываемое веб-приложение VetClinic предназначено для автоматизации основных процессов ветеринарной клиники и объединения данных о пациентах, приемах, лечении, складе, финансах и отчетности в единой информационной системе.

2.2 Проектирование базы данных

Для хранения данных веб-приложения VetClinic спроектирована реляционная база данных PostgreSQL (Рисунок 2.1). Проектирование выполнялось с использованием подхода Code First через Entity Framework Core. Такой подход позволяет описывать структуру базы данных в виде моделей C#, после чего создавать и обновлять таблицы с помощью миграций.

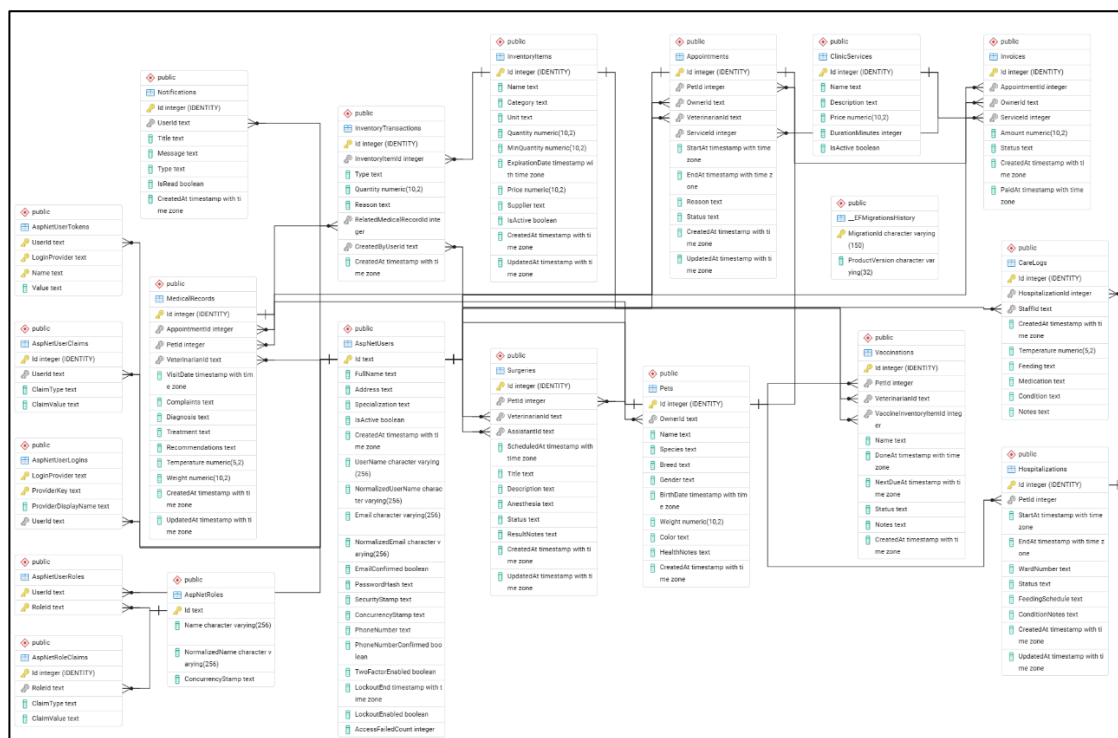


Рисунок 2.1 Схема базы данных

Основной таблицей системы является таблица пользователей, реализованная на основе ASP.NET Core Identity. Она предназначена для хранения учетных записей пользователей, их контактных данных, активности и дополнительной информации, необходимой для работы системы. Для разграничения доступа используются стандартные таблицы ролей и связей пользователей с ролями. В системе предусмотрены роли администратора, ветеринарного врача, владельца питомца и ассистента.

Для хранения информации о питомцах используется таблица Pet. Она содержит данные о животном: имя, вид, породу, пол, дату рождения, вес, цвет, особенности здоровья и идентификатор владельца. Между пользователем-владельцем и питомцами реализована связь «один ко многим», так как один владелец может иметь несколько питомцев.

Таблица Appointment предназначена для хранения записей на прием. Она связывает питомца, владельца, ветеринарного врача и выбранную услугу клиники. В таблице сохраняются дата и время начала приема, дата и время окончания, причина обращения, статус записи и служебные даты изменений.

Такая структура позволяет организовать расписание приемов и контролировать их состояние.

Для ведения медицинской истории используется таблица `MedicalRecord`. Она содержит сведения о приеме, питомце, ветеринарном враче, жалобах, диагнозе, лечении, рекомендациях, температуре и весе животного. Медицинская запись может быть связана с конкретной записью на прием, что позволяет сохранять результаты осмотра в истории болезней питомца.

Для учета вакцинаций предусмотрена таблица `Vaccination`. В ней хранятся данные о питомце, ветеринарном враче, названии вакцины, дате проведения вакцинации, дате следующей вакцинации, статусе и примечаниях. При необходимости вакцинация может быть связана со складской позицией, если вакцина учитывается на складе.

Складской учет реализуется с помощью таблиц `InventoryItem` и `InventoryTransaction`. Таблица `InventoryItem` хранит сведения о складских позициях: названии, категории, единице измерения, количестве, минимальном остатке, сроке годности, цене и поставщике. Таблица `InventoryTransaction` фиксирует операции движения товара: приход, списание, корректировку или расход. Это позволяет контролировать остатки лекарств, вакцин, кормов и расходных материалов.

Для операций и стационара используются таблицы `Surgery`, `Hospitalization` и `CareLog`. Таблица `Surgery` хранит информацию об операциях, включая питомца, врача, ассистента, дату, описание, наркоз, статус и результат. Таблица `Hospitalization` предназначена для учета пребывания питомца в стационаре, а `CareLog` хранит записи ухода, добавляемые сотрудниками клиники.

Финансовый блок реализован через таблицу `Invoice`. Она содержит сведения о счете, связанном с приемом, владельцем и услугой клиники. В таблице сохраняются сумма, статус оплаты, дата создания и дата оплаты. Для хранения услуг используется таблица `ClinicService`, содержащая название услуги, описание, цену, длительность и признак активности.

Для уведомлений предусмотрена таблица Notification. В ней хранятся получатель, заголовок, текст сообщения, тип уведомления, признак прочтения и дата создания. Эта таблица используется совместно с SignalR для отображения пользователям важных событий в режиме реального времени.

В результате проектирования разработана структура базы данных, обеспечивающая хранение основных данных ветеринарной клиники. Связи между таблицами позволяют корректно учитывать владельцев, питомцев, приемы, медицинскую историю, складские операции, финансовые данные и уведомления, а также обеспечивают целостность информации при работе приложения.

2.3 Реализация интерфейсов и функционала

В ходе разработки веб-приложения VetClinic реализована клиент-серверная архитектура, включающая серверную часть VetClinic.Api, клиентскую часть VetClinic.Client и общий проект VetClinic.Shared. Серверная часть отвечает за обработку запросов, выполнение бизнес-логики, авторизацию, работу с базой данных и отправку уведомлений. Клиентская часть предоставляет веб-интерфейс для взаимодействия с системой, а общий проект используется для хранения DTO-классов и перечислений.

Основная настройка серверного приложения выполняется в файле Program.cs. В нем подключаются ApplicationDbContext, ASP.NET Core Identity, JWT-аутентификация, SignalR, Swagger, CORS, контроллеры и сервисы приложения.

Для работы с базой данных реализован класс ApplicationDbContext, наследуемый от IdentityDbContext. В нем объявлены DbSet для основных сущностей проекта: питомцев, записей на прием, медицинских записей, вакцинаций, складских позиций, операций, госпитализаций, счетов и уведомлений (Рисунок 2.2).

```

public class AppDbContext : IdentityDbContext<ApplicationUser>
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Pet> Pets => Set<Pet>();
    public DbSet<Appointment> Appointments => Set<Appointment>();
    Ссылка: 7
    public DbSet<ClinicService> ClinicServices => Set<ClinicService>();
    Ссылка: 5
    public DbSet<MedicalRecord> MedicalRecords => Set<MedicalRecord>();
    Ссылка: 3
    public DbSet<Vaccination> Vaccinations => Set<Vaccination>();
    Ссылка: 13
    public DbSet<InventoryItem> InventoryItems => Set<InventoryItem>();
    Ссылка: 3
    public DbSet<InventoryTransaction> InventoryTransactions => Set<InventoryTransaction>();
    Ссылка: 2
    public DbSet<Surgery> Surgeries => Set<Surgery>();
    Ссылка: 7
    public DbSet<Hospitalization> Hospitalizations => Set<Hospitalization>();
    Ссылка: 2
    public DbSet<CareLog> CareLogs => Set<CareLog>();
    Ссылка: 5
    public DbSet<Invoice> Invoices => Set<Invoice>();
    Ссылка: 4
    public DbSet<Notification> Notifications => Set<Notification>();
}

```

Рисунок 2.2 AppDbContext.cs

Серверная часть построена с разделением контроллеров и сервисов. Контроллеры принимают HTTP-запросы и передают их в сервисный слой, где реализуется основная бизнес-логика приложения. Такой подход позволяет разграничить обработку запросов, работу с данными и прикладную логику системы.

Для авторизации пользователей реализован AuthService. Он выполняет регистрацию владельцев питомцев, регистрацию сотрудников администратором, вход в систему и получение данных текущего пользователя. При успешной авторизации формируется JWT-токен, который используется при обращении к защищенным API-методам. Разграничение доступа выполняется с помощью ролей Admin, Veterinarian, Owner и Assistant.

Функционал записи на прием реализован через AppointmentsController и AppointmentService. Контроллер содержит методы для получения записей, просмотра собственных приемов, получения записей врача, создания записи, изменения статуса и отмены приема. В сервисе выполняется проверка существования питомца, услуги и ветеринарного врача, проверяется принадлежность питомца текущему пользователю.

Затем рассчитывается время окончания приема по длительности услуги и проверяется, свободно ли выбранное время у врача.

После создания записи система отправляет уведомление ветеринарному врачу о новой записи на прием. При изменении статуса приема владелец получает уведомление об изменении состояния записи.

Отдельный функциональный блок предназначен для учета питомцев и ведения медицинских карт. Пользователь может управлять данными своих животных, а ветеринарный врач получает доступ к информации, необходимой для проведения приема. На основе посещений формируются медицинские записи, содержащие сведения о жалобах, диагнозе, лечении и рекомендациях, что позволяет хранить историю заболеваний питомцев в электронном виде.

В системе также реализован модуль вакцинации, который позволяет фиксировать сведения о проведенных прививках и отслеживать ближайшие или просроченные вакцинации. Это упрощает контроль профилактических мероприятий и повышает удобство ведения ветеринарного учета.

Складской учет реализован с использованием InventoryService. Он предназначен для учета медикаментов, вакцин, кормов и расходных материалов. В нем реализованы операции изменения остатков, а также контроль минимального количества и сроков годности. Дополнительно система позволяет выявлять критические остатки, что важно для своевременного пополнения запасов и стабильной работы клиники.

Для поддержки внутренних процессов клиники реализованы модули операций, стационара и финансового учета. Система позволяет сохранять сведения об операциях, вести учет госпитализации животных и формировать счета за оказанные услуги. Владельцы могут просматривать свои счета, а администратор получает доступ к финансовой информации в рамках своих полномочий.

Функции аналитики представлены модулем отчетности и административной панелью. В системе формируются данные по выручке, загрузке врачей, популярности услуг, расходу материалов и количеству приемов.

Ключевые показатели выводятся на административной панели, что позволяет оперативно оценивать текущее состояние работы клиники (Рисунок 2.3).

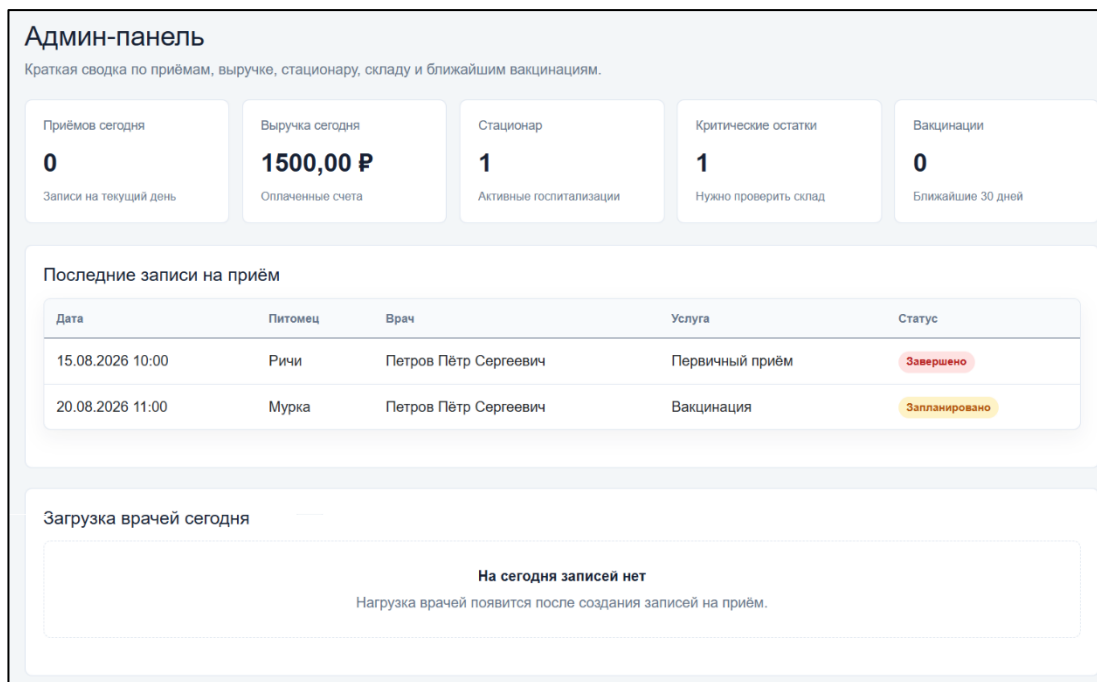


Рисунок 2.3 Административная панель

Для отправки уведомлений используется связка NotificationService и NotificationHub. Уведомления сохраняются в системе и передаются пользователям в режиме реального времени с помощью SignalR, что позволяет своевременно информировать владельцев, врачей и сотрудников об изменениях в работе приложения.

Клиентская часть приложения разработана на Blazor WebAssembly. В ней реализованы страницы для авторизации, работы с питомцами, записями на прием, медицинскими картами, вакцинациями, складом, отчетами, уведомлениями и административной панелью. Для взаимодействия с серверной частью используются клиентские сервисы, отправляющие HTTP-запросы к REST API. Навигация и доступные разделы интерфейса зависят от роли пользователя, что обеспечивает удобство работы и соблюдение разграничения прав доступа.

Таким образом, в проекте VetClinic реализованы основные интерфейсы и функциональные модули, необходимые для автоматизации работы ветеринарной клиники.

Приложение поддерживает авторизацию, ролевое управление доступом, ведение данных о питомцах и приемах, медицинский учет, складские операции, финансовый учет, отчетность и уведомления в режиме реального времени.

2.4 Тестирование программного продукта

После завершения разработки веб-приложения VetClinic проведено ручное функциональное тестирование. Проверка выполнялась для серверной и клиентской частей приложения, а также для взаимодействия с базой данных. Основное внимание уделялось корректности работы авторизации, разграничения доступа по ролям, CRUD-операций, записи на прием, медицинских карт, складского учета, финансового модуля и уведомлений.

Тестирование методом «черного ящика» проводилось для проверки функций приложения со стороны пользователя без анализа внутренней структуры кода. Проверены основные сценарии работы администратора, ветеринарного врача, владельца питомца и ассистента (Таблица 2.1).

Таблица 2.1 - Тест-кейсы для метода черного ящика

№	Что проверялось	Ожидаемый результат	Результат
1	Регистрация и авторизация пользователя	Пользователь успешно входит в систему	Успешно
2	Разграничение доступа по ролям	Пользователь видит только доступные ему разделы	Успешно
3	Добавление питомца владельцем	Питомец отображается в списке питомцев владельца	Успешно
4	Создание записи на прием	Запись сохраняется и отображается у владельца и врача	Успешно
5	Создание медицинской записи врачом	Запись появляется в истории болезней питомца	Успешно
6	Добавление вакцинации	Вакцинация отображается в списке и календаре вакцинаций	Успешно
7	Работа со складской позицией	Позиция создается, изменяется и отображается на складе	Успешно
8	Создание счета	Счет связывается с приемом и владельцем	Успешно
9	Просмотр отчетов администратором	Отображаются данные за выбранный период	Успешно
10	Получение уведомления	Уведомление отображается в интерфейсе пользователя	Успешно

Тестирование методом «белого ящика» проводилось для проверки внутренней логики приложения, работы сервисов, корректности запросов к базе данных и выполнения бизнес-правил. В ходе проверки анализировалась работа методов, отвечающих за создание записей на прием, проверку прав доступа, изменение складских остатков, формирование счетов и отправку уведомлений (Таблица 2.2).

Таблица 2.2 - Тест-кейсы для метода черного ящика

№	Что проверялось	Ожидаемый результат	Результат
1	Проверка JWT-авторизации	Защищенные API-методы недоступны без токена	Успешно
2	Проверка доступа по ролям	Пользователь с неподходящей ролью получает отказ в доступе	Успешно
3	Проверка принадлежности питомца владельцу	Владелец не может работать с чужими питомцами	Успешно
4	Проверка занятости времени врача	Повторная запись на занятое время не создается	Успешно
5	Расчет времени окончания приема	EndAt рассчитывается по длительности выбранной услуги	Успешно
6	Создание медицинской записи	Запись связывается с питомцем, врачом и приемом	Успешно
7	Изменение складского остатка	Количество товара изменяется по типу складской операции	Успешно
8	Проверка критического остатка	При низком количестве создается уведомление	Успешно
9	Создание счета по приему	Сумма счета соответствует стоимости выбранной услуги	Успешно
10	Отправка SignalR-уведомлений	Уведомление передается нужному пользователю	Успешно

Дополнительно выполнена проверка REST API через Swagger. Проверялись запросы на получение, создание, изменение и удаление данных, а также корректность возвращаемых HTTP-статусов. Для защищенных методов проверялось поведение без токена, с корректным токеном и с токеном пользователя, не имеющего нужной роли.

Также проверена работа клиентской части приложения. Тестирование включало переходы между страницами, отображение данных после загрузки с сервера, отправку форм, обработку ошибок и изменение навигационного меню в зависимости от роли пользователя. Отдельно проверялась работа страниц питомцев, записей на прием, медицинских карт, склада, счетов, отчетов и уведомлений.

По результатам тестирования установлено, что основные функции веб-приложения VetClinic работают корректно. Выявленные ошибки исправлены, после чего выполнена повторная проверка. Разработанное приложение стабильно выполняет основные пользовательские сценарии и корректно обрабатывает действия пользователей разных ролей.

2.5 Руководство пользователя

Для начала работы с веб-приложением VetClinic пользователь должен открыть главную страницу приложения и выполнить вход в систему (Рисунок 2.4). Если учетная запись отсутствует, владелец питомца может пройти регистрацию, указав необходимые данные. После успешной авторизации пользователю становятся доступны разделы, соответствующие его роли.

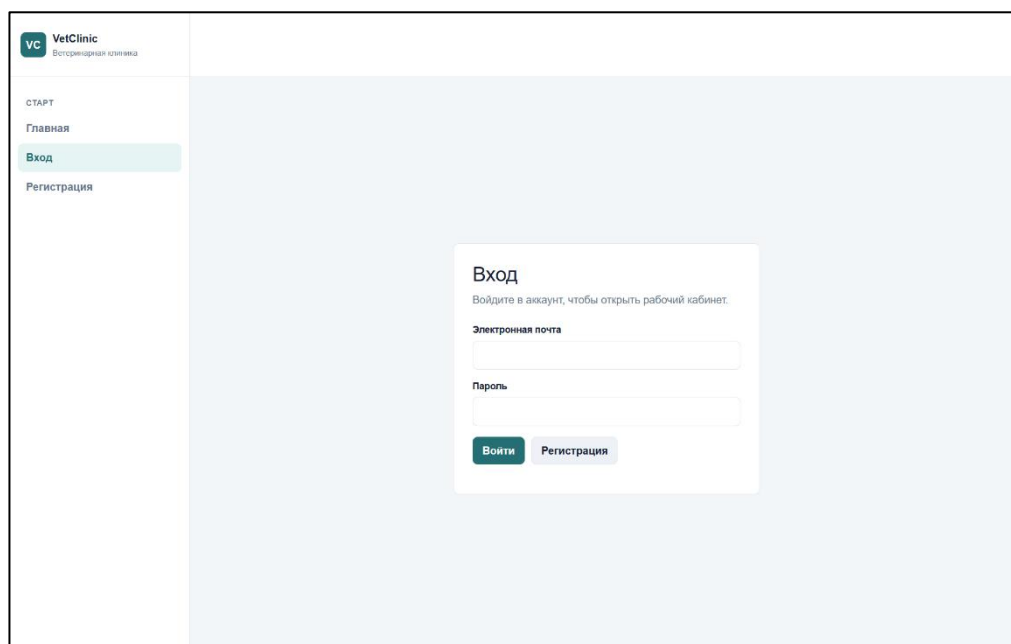


Рисунок 2.4 Страница авторизации

В системе предусмотрены четыре роли: администратор, ветеринарный врач, владелец питомца и ассистент. После входа навигационное меню формируется в зависимости от роли пользователя. Благодаря этому каждый пользователь видит только те разделы, которые необходимы для выполнения его задач.

Владелец питомца может перейти в раздел «Мои питомцы» и добавить карточку животного. Для создания карточки необходимо указать имя питомца, вид, породу, пол, дату рождения, вес, цвет и особенности здоровья. После сохранения питомец отображается в списке, а его данные можно просмотреть или изменить (Рисунок 2.5).

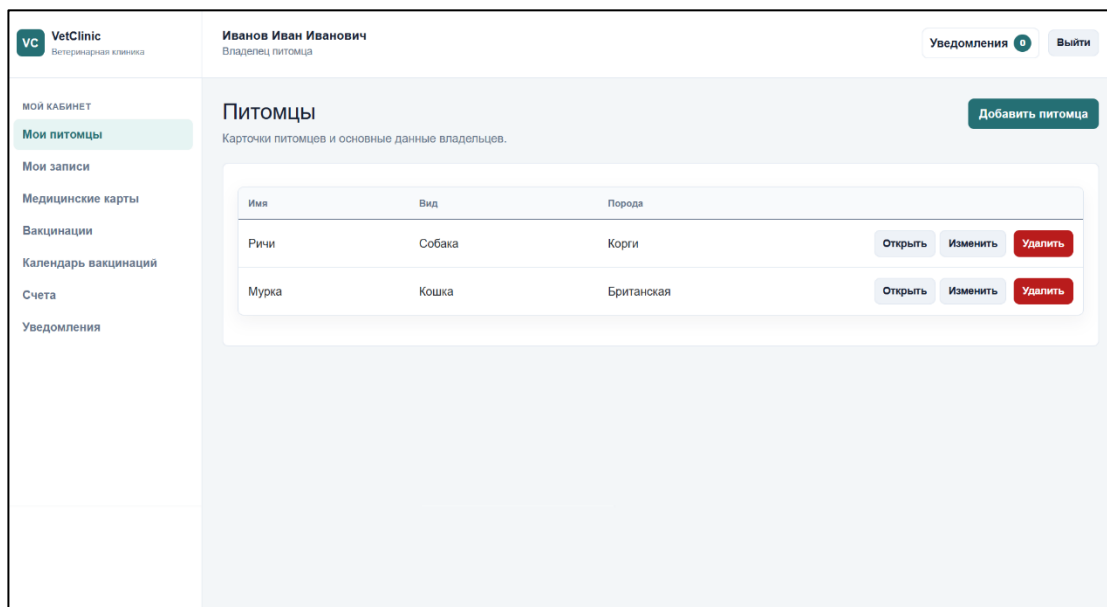


Рисунок 2.5 Страница со список питомцев владельца

Для записи питомца на прием владелец переходит в раздел создания записи. В форме необходимо выбрать питомца, ветеринарного врача, услугу клиники, дату и время приема, а также указать причину обращения. После подтверждения запись сохраняется в системе и отображается в списке приемов владельца. Ветеринарный врач также получает доступ к созданной записи в своем разделе приемов.

Ветеринарный врач работает с разделом «Мои приемы». В нем отображаются записи, назначенные конкретному врачу. После проведения приема врач может создать медицинскую запись, указав жалобы, диагноз, лечение, рекомендации, температуру и вес питомца (Рисунок 2.6). Созданная запись сохраняется в истории болезней животного и может быть просмотрена в дальнейшем.

vc VetClinic Ветеринарная клиника

Петров Пётр Сергеевич
Ветеринарный врач

Уведомления 0 Выйти

РАБОТА ВРАЧА

Мои записи

Питомцы

Медицинские карты

Вакцинации

Календарь вакцинаций

Операции

Стационар

Склад

Уведомления

Новая медицинская запись

Создайте запись осмотра по выбранному питомцу и приёму.

Питомец
Ричи

Приём
Без приёма

Дата визита
08.05.2026

Температура
Например: 38,5

Вес
Вес в кг

Жалобы
Опишите жалобы владельца

Диагноз
Укажите диагноз

Лечение и назначения
Назначения, препараты и план лечения

Рекомендации
Рекомендации владельцу

Создать **Назад**

Рисунок 2.6 Форма создания медицинской записи

В разделе вакцинаций врач может добавить сведения о проведенной или запланированной вакцинации. Для этого указываются питомец, название вакцины, дата проведения, дата следующей вакцинации, статус и примечания. Владелец питомца может просматривать вакцинации своих животных и получать уведомления о предстоящих профилактических мероприятиях (Рисунок 2.7).

vc VetClinic Ветеринарная клиника

Иванов Иван Иванович
Владелец питомца

Уведомления 0 Выйти

Мой кабинет

Мои питомцы

Мои записи

Медицинские карты

Вакцинации

Календарь вакцинаций

Счета

Уведомления

Вакцинации

История вакцинаций питомцев и план следующих дат.

Питомец
Все питомцы

Питомец	Вакцина	Дата вакцинации	Следующая дата	Статус	Врач	Заметки
Мурка	Рабизин	01.07.2026	01.08.2026	Просрочено	Петров Пётр Сергеевич	Нужно пригласить владельца на повторную вакцинацию
Ричи	Нобивак DHPPi	15.08.2026	15.09.2026	Завершено	Петров Пётр Сергеевич	Плановая вакцинация

Рисунок 2.7 Страница вакцинаций питомца

Администратор имеет доступ к административным разделам системы. Он может просматривать пользователей, владельцев, питомцев, приемы, склад, счета, отчеты и административную панель.

В административной панели отображаются основные показатели работы клиники: количество приемов за текущий день, выручка, активные госпитализации, критические остатки на складе, ближайшие вакцинации и последние записи на прием.

Для работы со складом администратор или ассистент переходит в раздел «Склад» (Рисунок 2.8). В этом разделе отображаются лекарственные препараты, вакцины, корма и расходные материалы. Пользователь с соответствующими правами может добавить новую складскую позицию, изменить ее данные, а также оформить приход, расход, списание или корректировку остатка. При низком количестве товара система формирует уведомление для администратора.

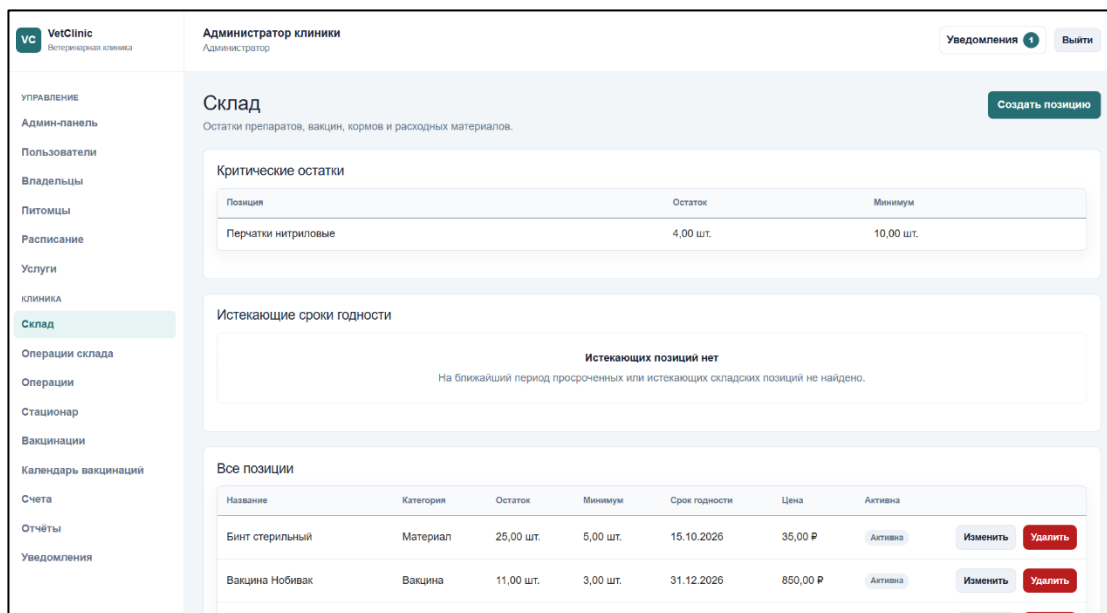


Рисунок 2.8 Страница склада

Ассистент также работает с разделом стационара. В нем отображаются активные госпитализации питомцев. При необходимости ассистент может добавить запись ухода, указав температуру, кормление, назначенные медикаменты, состояние животного и дополнительные примечания.

Финансовый раздел предназначен для работы со счетами. Администратор может создать счет на основании приема и выбранной услуги, а также отметить его как оплаченный. Владелец питомца может просматривать только свои счета и их текущий статус.

В разделе отчетов администратор может получить сводную информацию о работе клиники. Система предусматривает отчеты по выручке за период, загрузке врачей, популярности услуг, расходу материалов и количеству приемов. Эти данные используются для анализа деятельности ветеринарной клиники.

Уведомления доступны пользователю в отдельном разделе и через компонент уведомлений в интерфейсе (Рисунок 2.9). Система может уведомлять владельца о приеме или вакцинации, врача - о новой записи, администратора - о критических остатках, а ассистента - об изменениях в стационаре. Уведомления передаются в режиме реального времени через SignalR и сохраняются в базе данных.

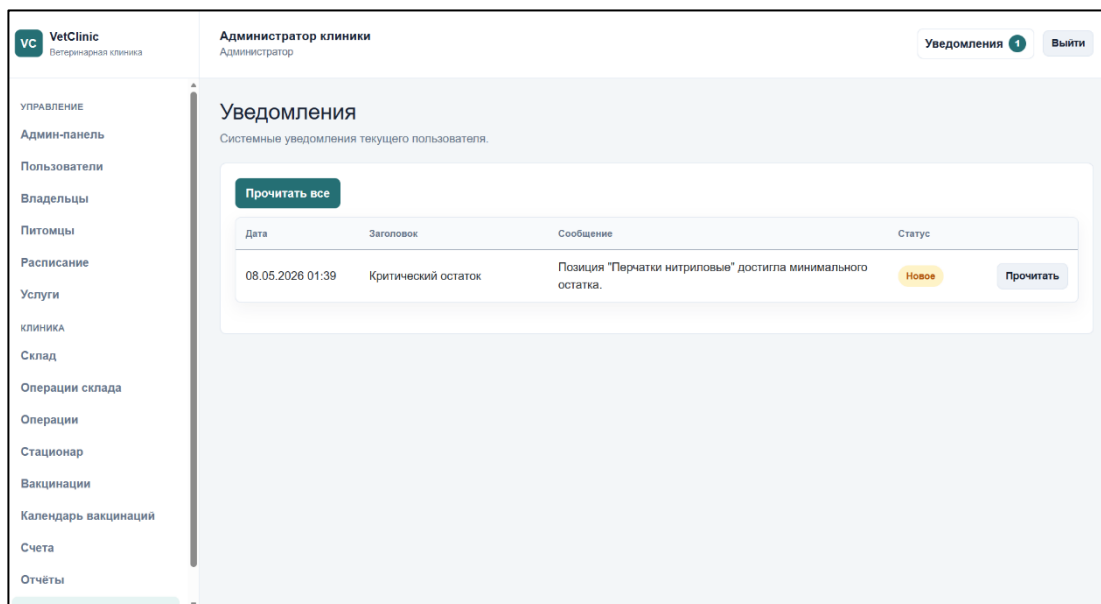


Рисунок 2.7 Страница вакцинаций питомца

После завершения работы пользователь может выйти из системы с помощью пункта «Выход» в навигационном меню. Разработанное веб-приложение VetClinic обеспечивает удобную работу с основными процессами ветеринарной клиники и позволяет пользователям выполнять действия в соответствии с назначенной ролью.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы разработано веб-приложение VetClinic, предназначенное для автоматизации основных процессов ветеринарной клиники. Созданная система позволяет вести учет владельцев питомцев, животных, записей на прием, медицинских карт, вакцинаций, складских позиций, финансовых счетов, отчетов и уведомлений.

В процессе разработки спроектирована структура приложения, включающая серверную часть VetClinic.Api с использованием ASP.NET Core Web API, клиентскую часть VetClinic.Client с использованием Blazor WebAssembly и общий проект VetClinic.Shared. Взаимодействие с базой данных выполняется через Entity Framework Core и СУБД PostgreSQL.

В системе реализовано разграничение доступа по ролям. Для администратора предусмотрены функции управления пользователями, питомцами, складом, финансами, отчетами и административной панелью. Ветеринарный врач может работать с приемами, медицинскими картами, вакцинациями и операциями. Владелец питомца может добавлять своих животных, записывать их на прием, просматривать историю болезней, вакцинации и счета. Ассистенту доступны функции работы со складом, стационаром и записями ухода.

Дополнительно в приложении реализованы уведомления в режиме реального времени с использованием SignalR. Они позволяют информировать пользователей о новых записях, изменениях статусов, предстоящих вакцинациях, критических остатках на складе и событиях стационара. Для администратора также предусмотрены отчеты и панель с основными показателями работы клиники.

Разработанное веб-приложение VetClinic обеспечивает удобную работу с данными ветеринарной клиники и демонстрирует полный цикл взаимодействия пользователей с системой. Полученное решение соответствует поставленному индивидуальному заданию и может использоваться как основа для дальнейшего развития проекта.